Task 1: Create VPC

1. Objective

To create a custom Virtual Private Cloud (VPC) in AWS using Terraform.

2. Steps Used

1. Created a fresh project directory `Final-Task`.
2. Created `main.tf`.
3. Configured AWS provider with `us-east-1`.
4. Wrote VPC resource configuration.
5. Executed `terraform init`.
6. Executed `terraform validate`.
7. Executed `terraform plan`.
8. Executed `terraform apply`.
9. Approved creation by entering `yes`.

3. Code

```
provider "aws" {
  region = "us-east-1"
}

resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"

  enable_dns_support   = true
  enable_dns_hostnames = true

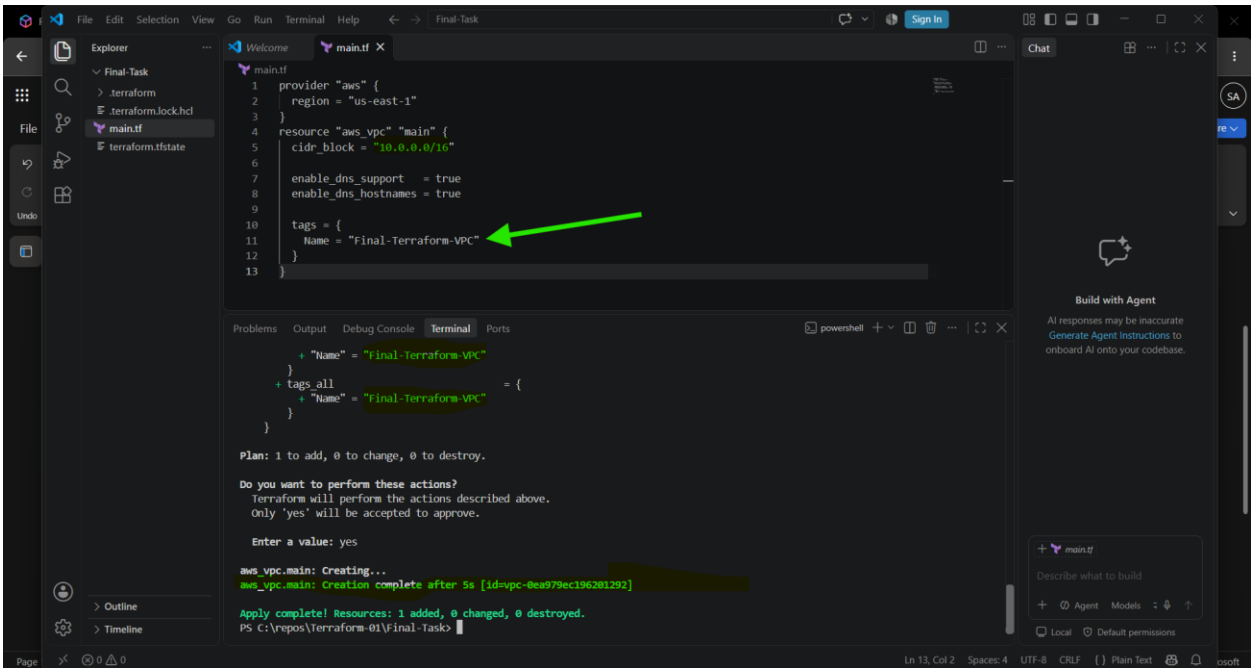
  tags = {
    Name = "Final-Terraform-VPC"
  }
}
```

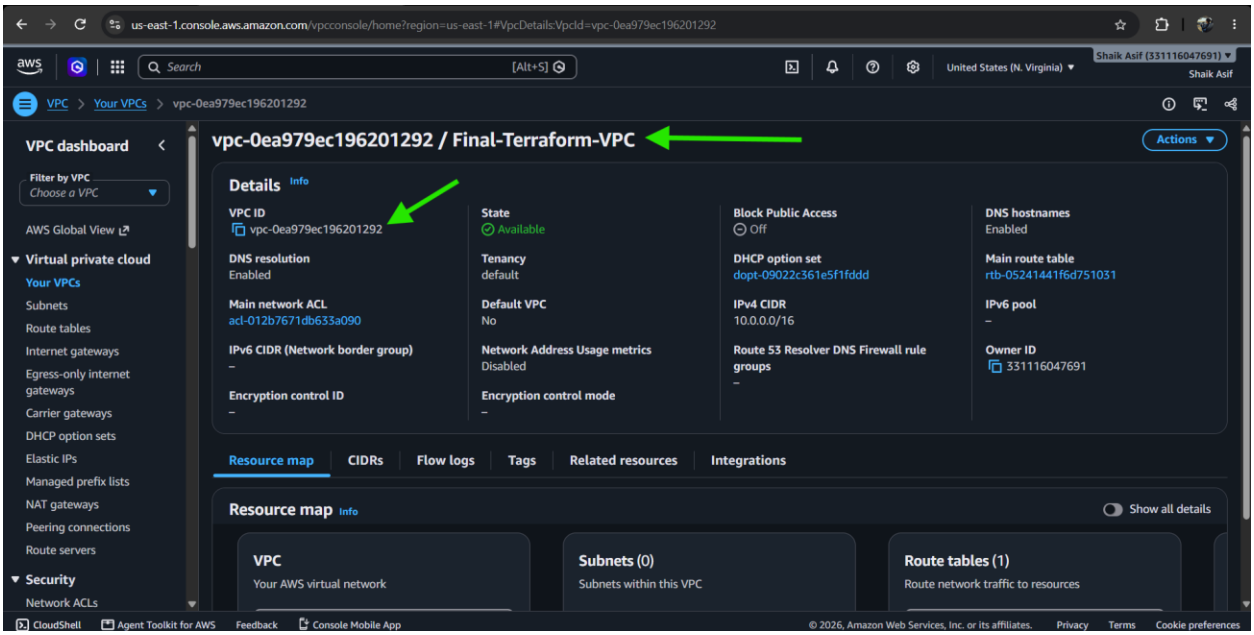
4. Why do we need VPC?

VPC provides an isolated networking environment in AWS.

We need it to:

- Create our own IP address range.
- Launch EC2 instances.
- Create public and private subnets.
- Control network communication.
- Connect resources to the internet through an Internet Gateway.





5. Observation

Terraform output:

```
aws_vpc.main: Creation complete
[id=vpc-0ea979ec196201292]

Apply complete!
Resources: 1 added, 0 changed, 0 destroyed.
```

6. Conclusion

Successfully created a custom VPC with CIDR `10.0.0.0/16` in AWS using Terraform. DNS support and DNS hostnames are enabled.

Task 2: Create Internet gateway

Objective

To create an Internet Gateway and attach it to our custom VPC using Terraform.

Steps Used

1. Opened the existing `main.tf`.
2. Added the Internet Gateway resource.
3. Referenced the VPC using `aws_vpc.main.id`.
4. Executed `terraform validate`.
5. Executed `terraform plan`.
6. Executed `terraform apply`.
7. Approved the execution by entering `yes`.

Code

Complete code used so far:

```
provider "aws" {
  region = "us-east-1"
}

# Task 1: Create VPC
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"

  enable_dns_support   = true
  enable_dns_hostnames = true

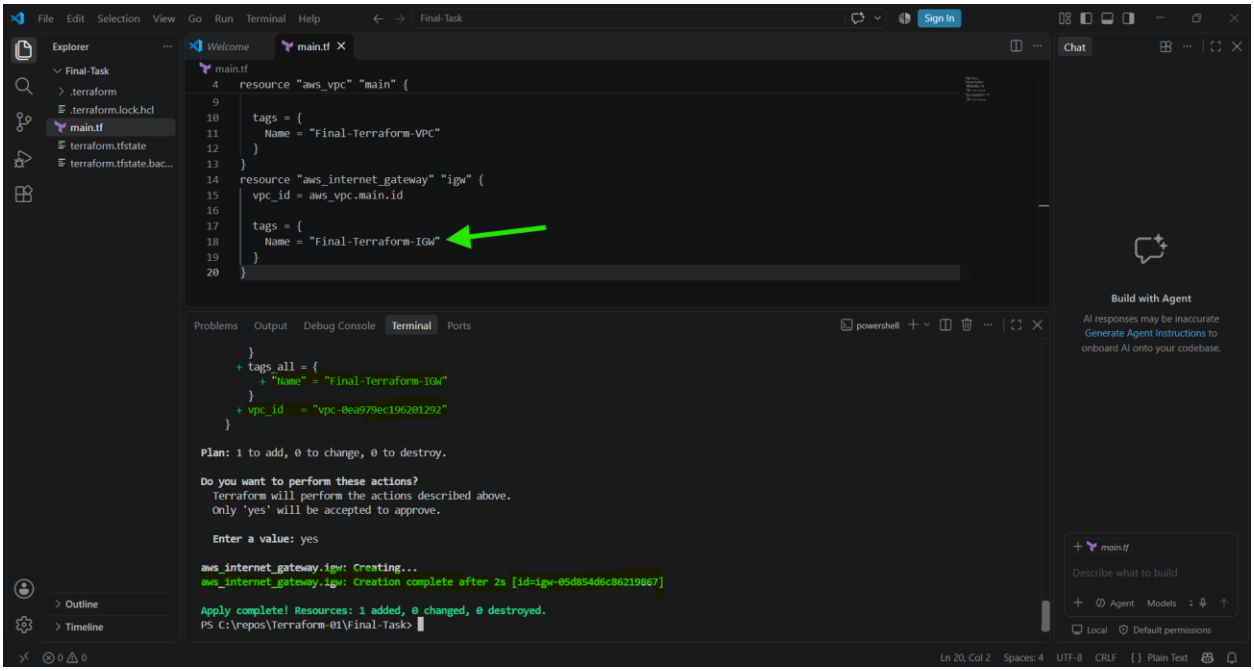
  tags = {
    Name = "Final-Terraform-VPC"
  }
}

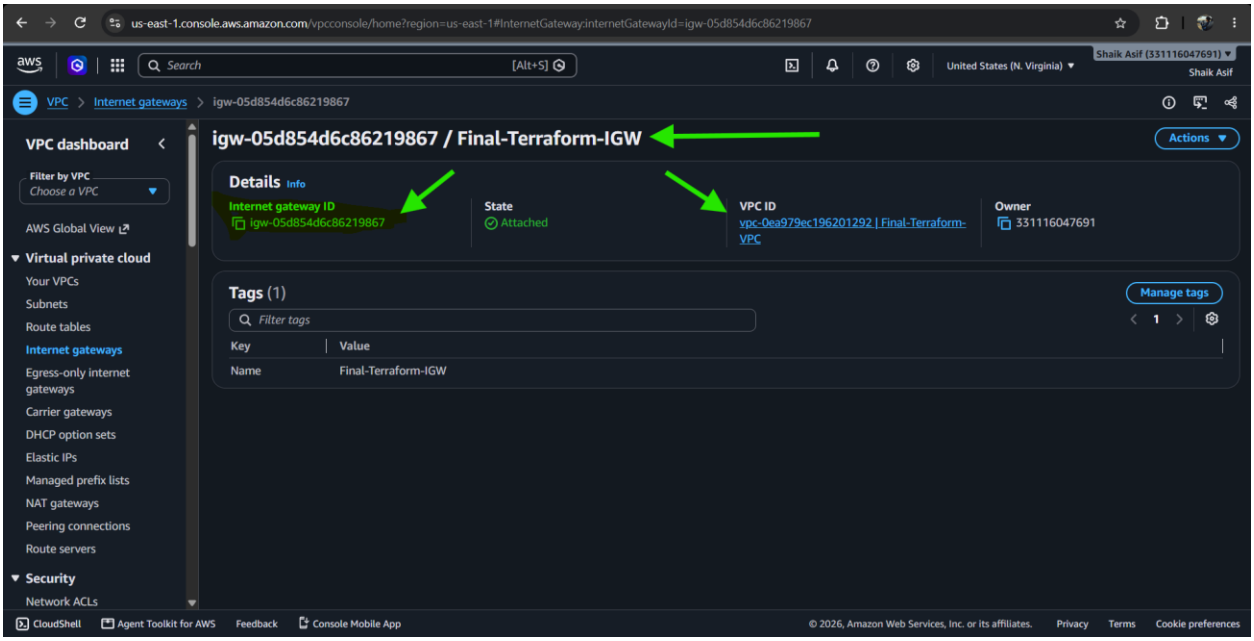
# Task 2: Create Internet Gateway

resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id

  tags = {
```

```
    Name = "Final-Terraform-IGW"
  }
}
```





Why do we need an Internet Gateway?

- Connects our VPC to the internet.

- Allows internet communication when routing and security rules permit it.
- Supports inbound and outbound internet connectivity for public resources.

Important: Creating an IGW alone does not make a subnet public. We also need a route table with a route to the IGW.

Observation

Terraform output:

```
aws_internet_gateway.igw: Creation complete
[id=igw-05d854d6c86219867]

Apply complete!
Resources: 1 added, 0 changed, 0 destroyed.
```

Conclusion

Successfully created an Internet Gateway and attached it to our custom VPC using Terraform.

Task 3: Create Custom Route Table

Objective

To create a custom route table inside our VPC and configure a default route that directs internet-bound IPv4 traffic through the Internet Gateway.

Steps Used

1. Opened the existing `main.tf`.
2. Added the custom route table resource.
3. Referenced the existing VPC using `aws_vpc.main.id`.
4. Configured the default route `0.0.0.0/0`.
5. Connected the route to the Internet Gateway.
6. Executed `terraform validate`.
7. Executed `terraform plan`.
8. Executed `terraform apply`.
9. Approved the changes by entering `yes`.

Code

Complete code used so far:

```
provider "aws" {
  region = "us-east-1"
}

# Task 1: Create VPC
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"

  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = {
    Name = "Final-Terraform-VPC"
  }
}

# Task 2: Create Internet Gateway

resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id

  tags = {
    Name = "Final-Terraform-IGW"
  }
}

# Task 3: Create Custom Route Table

resource "aws_route_table" "public_rt" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }
```

```
  tags = {
    Name = "Final-Terraform-Public-RT"
  }
}
```

Why do we need a Custom Route Table?

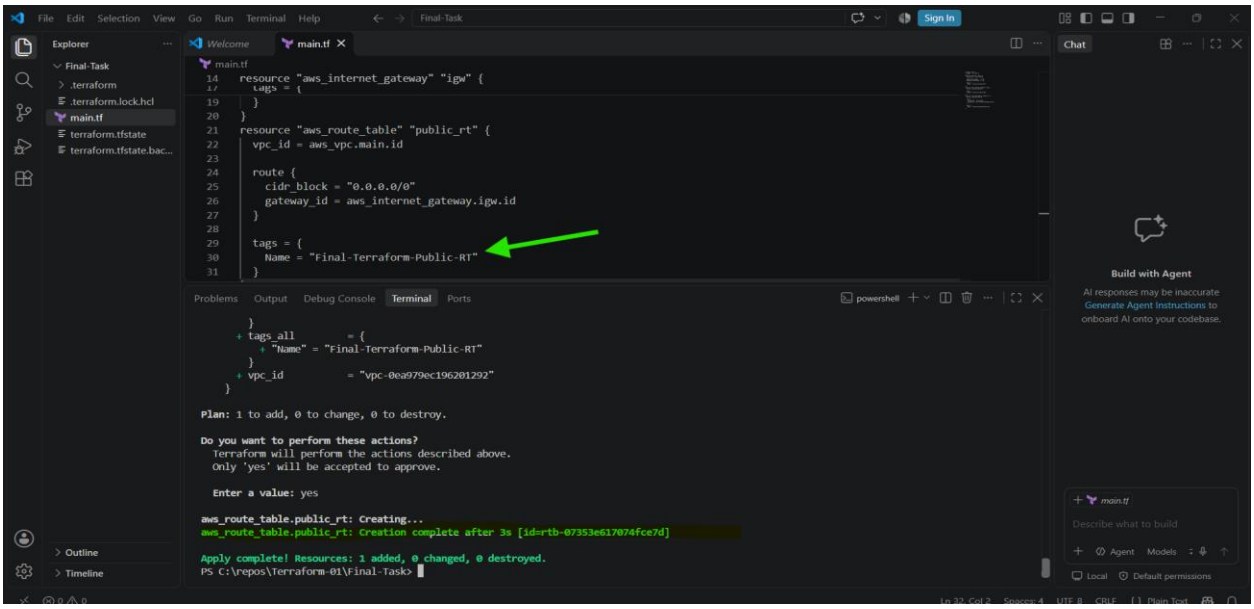
A route table determines where network traffic should go.

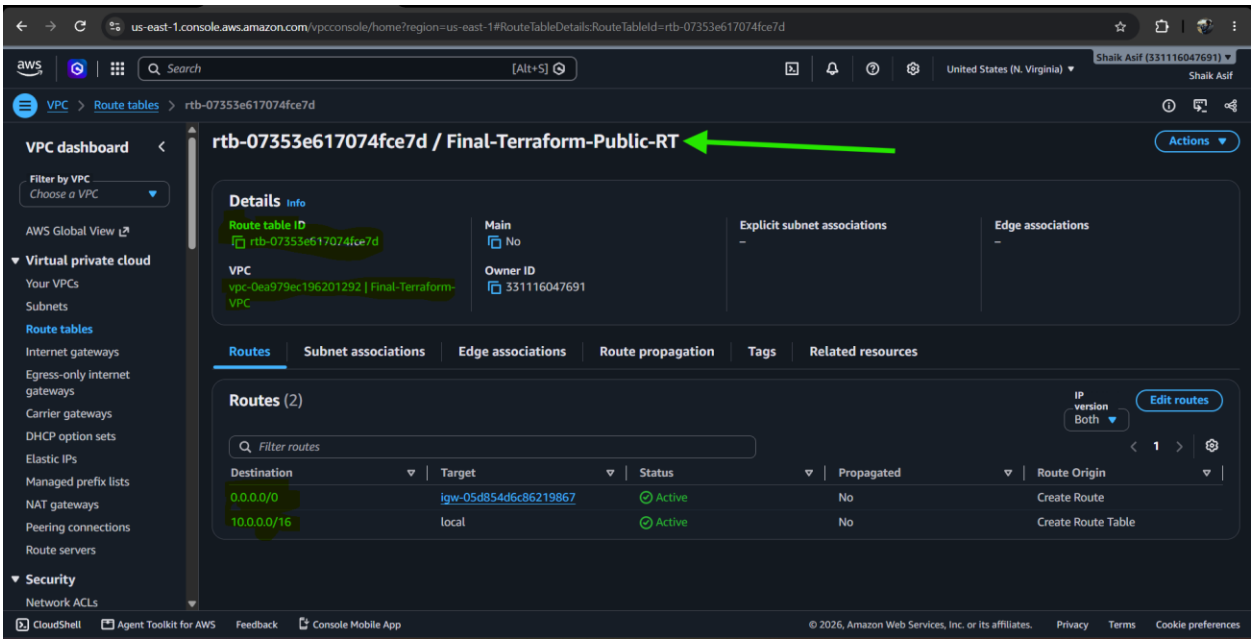
For our project:

- Local VPC traffic stays within the VPC.
- Internet-bound traffic (`0.0.0.0/0`) goes through the Internet Gateway.
- This provides the routing configuration needed for public subnet connectivity.

Important concepts:

- `0.0.0.0/0` = All IPv4 destinations.
- `gateway_id` = Destination gateway.
- `aws_internet_gateway.igw.id` = Automatically retrieves our IGW ID.





Observation

Your Terraform output:

```
aws_route_table.public_rt: Creation complete
[id=rtb-07353e617074fce7d]

Apply complete!
Resources: 1 added, 0 changed, 0 destroyed.
```

Successfully created

Conclusion

Successfully created a custom route table inside our VPC and configured a default IPv4 route through the Internet Gateway.

Task 4: Create Subnet

Objective

To create a subnet inside our custom VPC using Terraform.

Steps Used

1. Opened the existing `main.tf`.
2. Added the subnet resource configuration.
3. Referenced the existing VPC using `aws_vpc.main.id`.
4. Configured subnet CIDR as `10.0.1.0/24`.
5. Selected Availability Zone `us-east-1a`.
6. Enabled automatic public IPv4 assignment.
7. Executed `terraform validate`.
8. Executed `terraform plan`.
9. Executed `terraform apply`.
10. Approved the creation by entering `yes`.

Code

Complete code used so far:

```
provider "aws" {
  region = "us-east-1"
}

# Task 1: Create VPC
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"

  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = {
    Name = "Final-Terraform-VPC"
  }
}

# Task 2: Create Internet Gateway

resource "aws_internet_gateway" "igw" {
```

```
  vpc_id = aws_vpc.main.id

  tags = {
    Name = "Final-Terraform-IGW"
  }
}

# Task 3: Create Custom Route Table

resource "aws_route_table" "public_rt" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }

  tags = {
    Name = "Final-Terraform-Public-RT"
  }
}

# Task 4: Create Subnet

resource "aws_subnet" "public_subnet" {
  vpc_id                  = aws_vpc.main.id
  cidr_block              = "10.0.1.0/24"
  availability_zone       = "us-east-1a"
  map_public_ip_on_launch = true

  tags = {
    Name = "Final-Terraform-Public-Subnet"
  }
}
```

Why do we need a Subnet?

A subnet divides a VPC into smaller network sections.

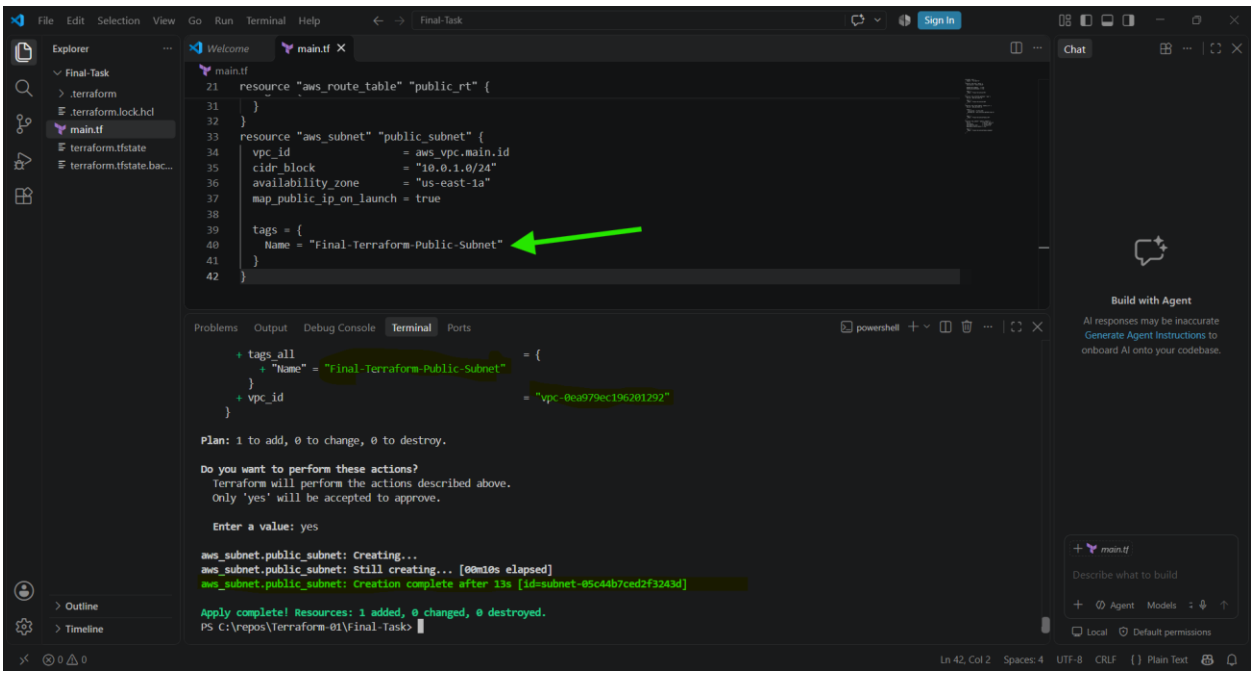
We need it to:

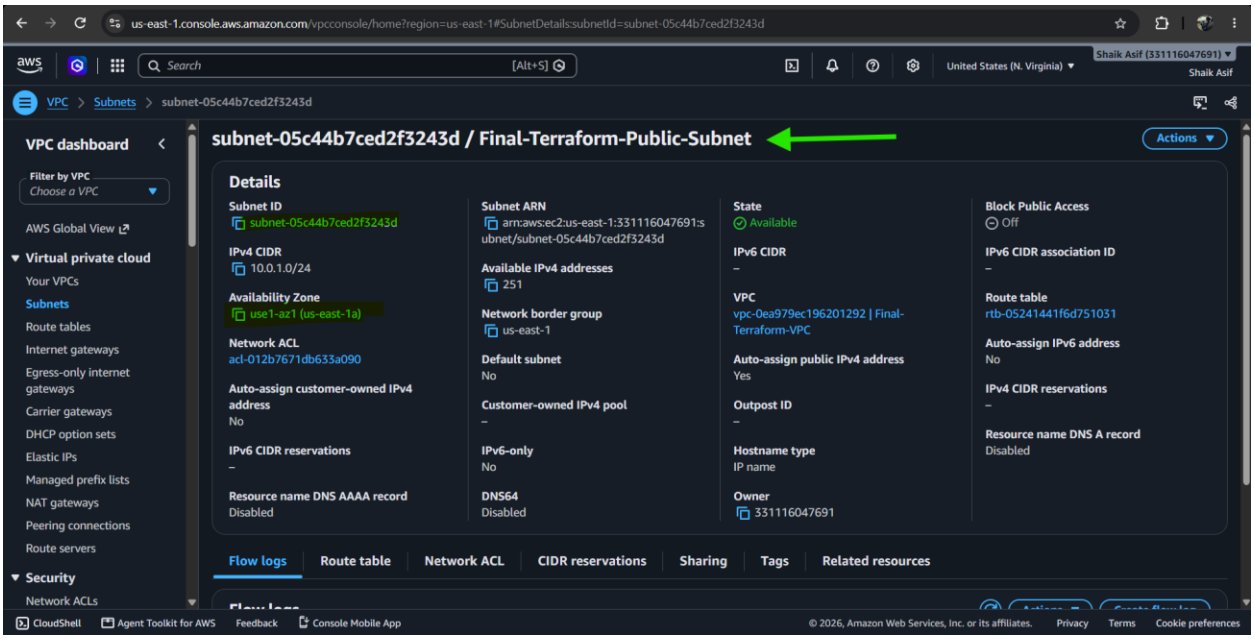
- Launch EC2 instances inside our VPC.
- Organize network resources.
- Control network traffic.
- Assign private IP addresses.
- Connect resources to route tables.

Important concept:

`map_public_ip_on_launch = true`

This automatically assigns public IPv4 addresses to eligible EC2 instances launched in this subnet. It does not itself make the subnet public.





Observation

Your Terraform output:

```
aws_subnet.public_subnet: Creation complete after 13s
[id=subnet-05c44b7ced2f3243d]

Apply complete!
Resources: 1 added, 0 changed, 0 destroyed.
```

Conclusion

Successfully created a subnet inside our custom VPC with CIDR `10.0.1.0/24` in Availability Zone `us-east-1a`.

Task 5: Associate subnet with Route Table

1. Objective

To associate the public subnet with the custom route table so that the subnet can use the Internet Gateway to communicate with the internet.

2. Steps Used

1. Opened PowerShell.
2. Changed the directory to the Terraform project:

```
cd C:\repos\Terraform-01\Final-Task
```

3. Added the route table association resource to `main.tf`.
4. Validated the Terraform configuration:

```
terraform validate
```

5. Checked the execution plan:

```
terraform plan
```

6. Applied the configuration:

```
terraform apply
```

7. Entered `yes` to confirm the changes.
8. Verified the association using AWS CLI:

```
aws ec2 describe-route-tables --route-table-ids rtb-07353e617074fce7d
--region us-east-1 --query
"RouteTables[0].Associations[*].[SubnetId,Main,AssociationState.State]
" --output table
```

3. Complete Code – Tasks 1 to 5

```
provider "aws" {
  region = "us-east-1"
}
```

```
# Task 1: Create VPC
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"

  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = {
    Name = "Final-Terraform-VPC"
  }
}

# Task 2: Create Internet Gateway
resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id

  tags = {
    Name = "Final-Terraform-IGW"
  }
}

# Task 3: Create Custom Route Table
resource "aws_route_table" "public_rt" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }

  tags = {
    Name = "Final-Terraform-Public-RT"
  }
}

# Task 4: Create Public Subnet
resource "aws_subnet" "public_subnet" {
  vpc_id                  = aws_vpc.main.id
```

```
  cidr_block              = "10.0.1.0/24"
  availability_zone       = "us-east-1a"
  map_public_ip_on_launch = true

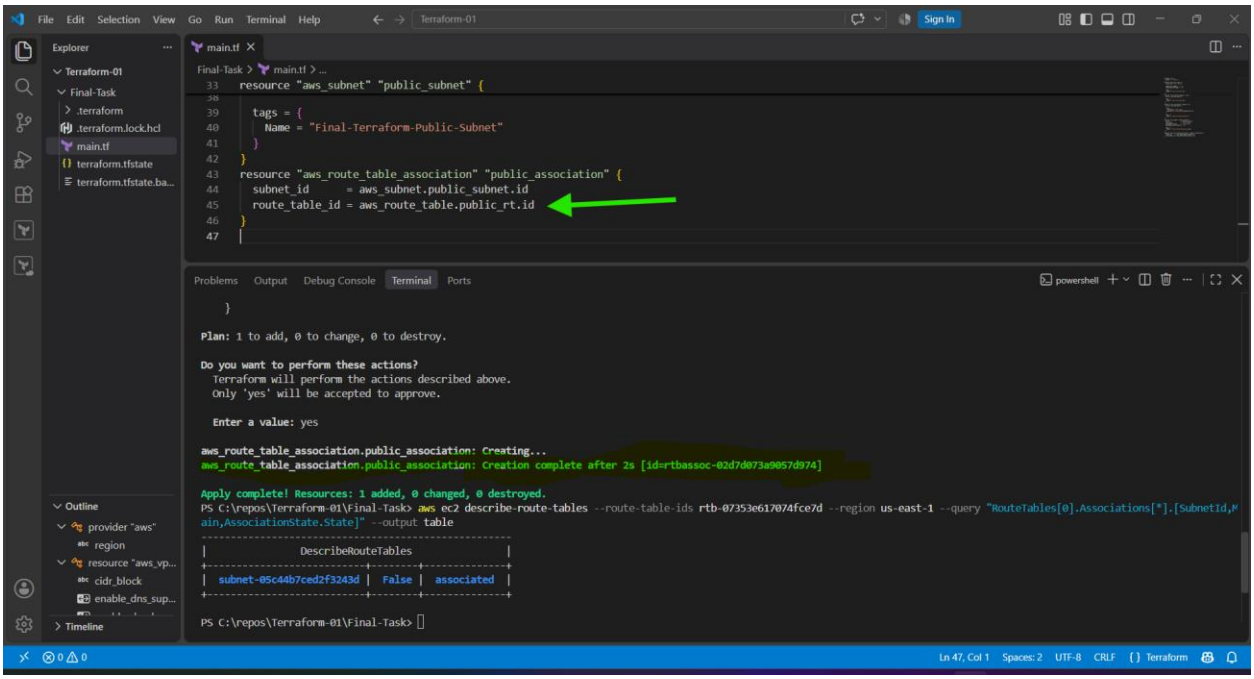
  tags = {
    Name = "Final-Terraform-Public-Subnet"
  }
}

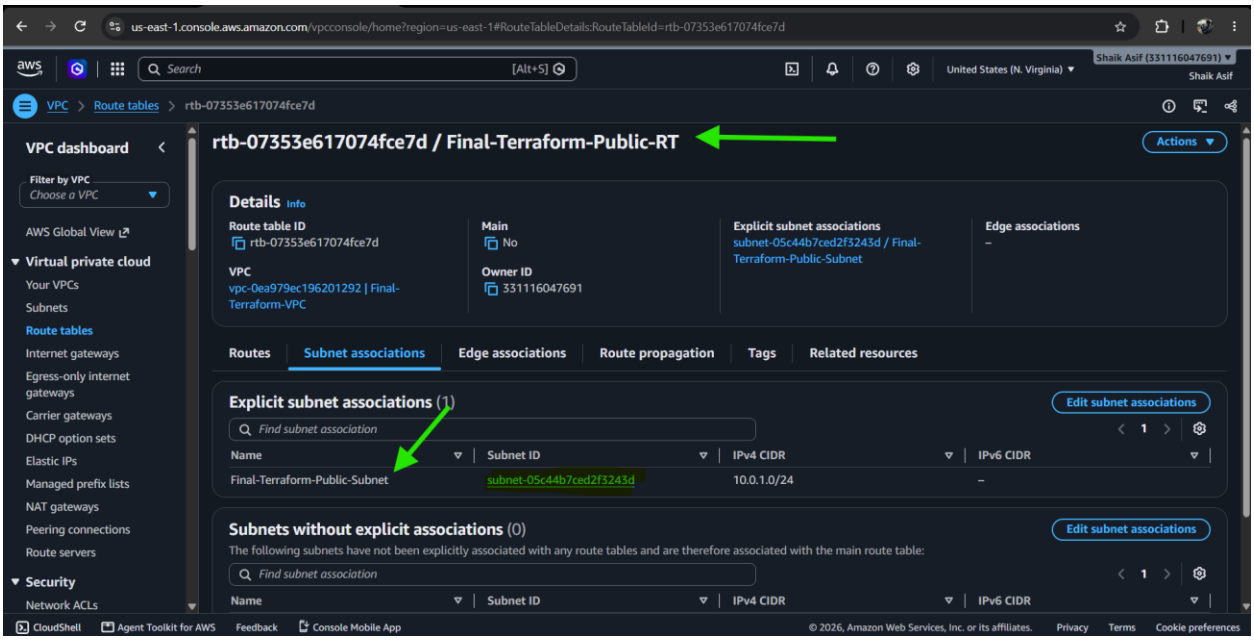
# Task 5: Associate Subnet with Route Table
resource "aws_route_table_association" "public_association" {
  subnet_id      = aws_subnet.public_subnet.id
  route_table_id = aws_route_table.public_rt.id
}
```

4. Why Do We Need This?

- A VPC can contain multiple subnets.
- A route table contains routing rules.
- A subnet must be associated with a route table to use its routes.
- Our custom route table has a default route `0.0.0.0/0` pointing to the Internet Gateway.
- By associating the subnet, we enable it to use that internet route.

Simple example: Route table is like a road map. Subnet association connects the subnet to that road map.





5. Observation

Terraform output:

```
aws_route_table_association.public_association:
Creation complete after 2s
```

```
Apply complete!
Resources: 1 added, 0 changed, 0 destroyed.
```

AWS CLI verification:

```
subnet-05c44b7ced2f3243d | False | associated
```

Created association ID:

```
rtbassoc-02d7d073a9057d974
```

6. Conclusion

Successfully associated the public subnet with the custom route table using Terraform. Verified the association through AWS CLI. The subnet can now use the Internet Gateway route configured in the route table.

Task 6: Create Security Group to allow port 22,80,443

1. Objective

To create a Security Group inside our VPC that allows incoming SSH, HTTP, and HTTPS traffic to the EC2 instance.

2. Steps Used

1. Opened PowerShell.
2. Navigated to the Terraform project directory:

```
cd C:\repos\Terraform-01\Final-Task
```

3. Added the Security Group resource to `main.tf`.
4. Executed Terraform plan:

```
terraform plan
```

5. Checked the plan: one resource to add.
6. Executed Terraform apply:

```
terraform apply
```

7. Entered `yes` to approve the creation.
8. Terraform created the Security Group successfully.

3. Complete Code – Tasks 1 to 6

```
provider "aws" {
  region = "us-east-1"
}

# Task 1: Create VPC
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"

  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = {
    Name = "Final-Terraform-VPC"
  }
}

# Task 2: Create Internet Gateway
resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id

  tags = {
    Name = "Final-Terraform-IGW"
  }
}

# Task 3: Create Custom Route Table
resource "aws_route_table" "public_rt" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
```

```
    gateway_id = aws_internet_gateway.igw.id
  }

  tags = {
    Name = "Final-Terraform-Public-RT"
  }
}

# Task 4: Create Public Subnet
resource "aws_subnet" "public_subnet" {
  vpc_id                  = aws_vpc.main.id
  cidr_block              = "10.0.1.0/24"
  availability_zone       = "us-east-1a"
  map_public_ip_on_launch = true

  tags = {
    Name = "Final-Terraform-Public-Subnet"
  }
}

# Task 5: Associate Subnet with Route Table
resource "aws_route_table_association" "public_association" {
  subnet_id      = aws_subnet.public_subnet.id
  route_table_id = aws_route_table.public_rt.id
}

# Task 6: Create Security Group
resource "aws_security_group" "web_sg" {
  name        = "Final-Terraform-SG"
  description = "Allow SSH, HTTP and HTTPS traffic"
  vpc_id      = aws_vpc.main.id

  # Allow SSH
  ingress {
    description = "Allow SSH"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
```

```
  }

  # Allow HTTP
  ingress {
    description = "Allow HTTP"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  # Allow HTTPS
  ingress {
    description = "Allow HTTPS"
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  # Allow all outbound traffic
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "Final-Terraform-SG"
  }
}
```

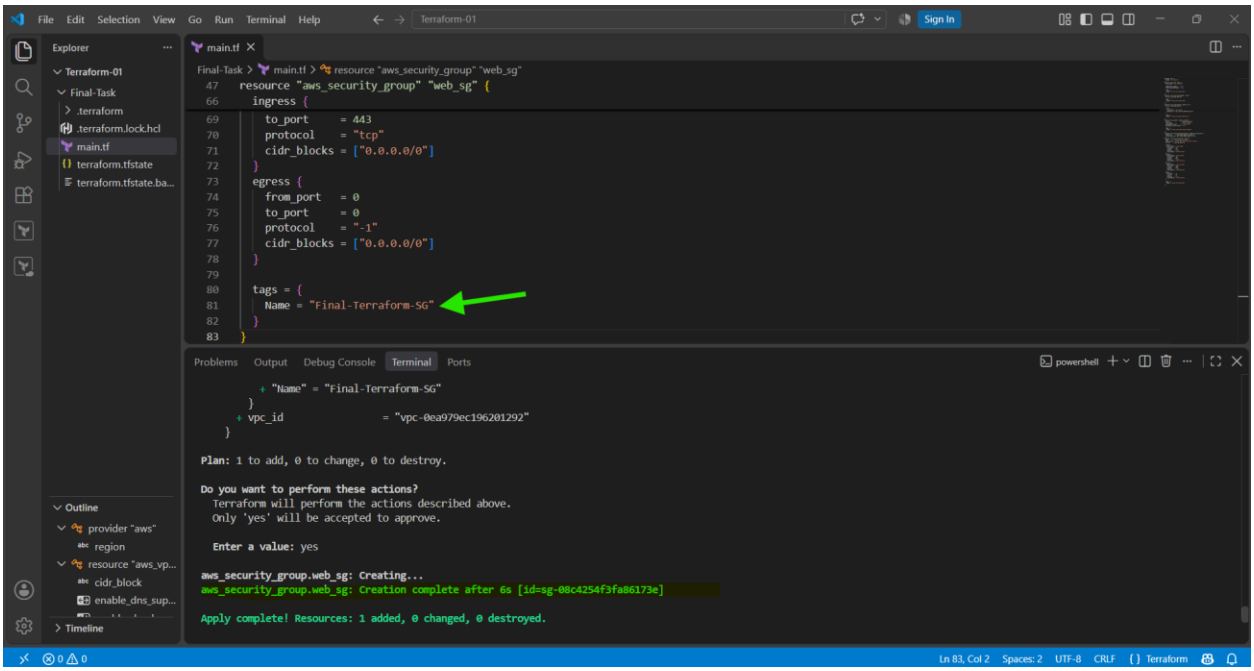
4. Why Do We Need This?

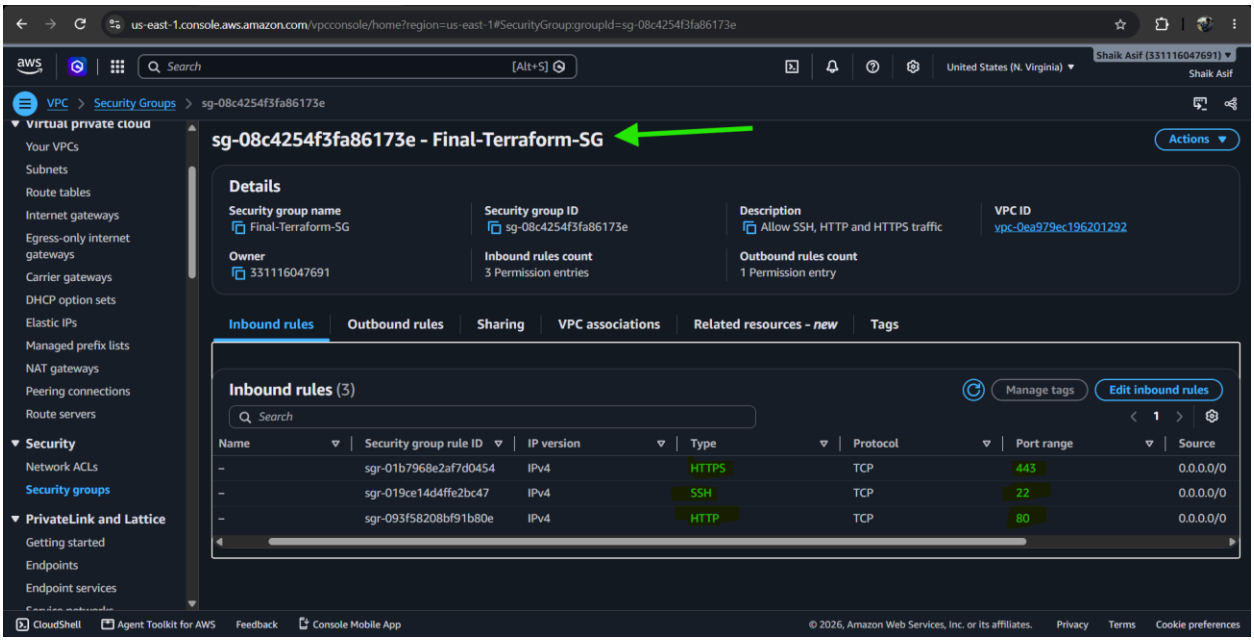
A Security Group acts as a virtual firewall for AWS resources.

Port	Service	Purpose
22	SSH	Remotely connect to Ubuntu EC2
80	HTTP	Access the Apache website
443	HTTPS	Access the secure website

Important Terraform arguments:

- `ingress` – controls incoming traffic.
- `egress` – controls outgoing traffic.
- `from_port` – starting port number.
- `to_port` – ending port number.
- `protocol = "tcp"` – allows TCP traffic.
- `protocol = "-1"` – allows all protocols.
- `cidr_blocks` – specifies which IP addresses are allowed.
- `0.0.0.0/0` – represents all IPv4 addresses.





5. Observation

Terraform plan:

```
Plan: 1 to add, 0 to change, 0 to destroy.
```

Terraform apply:

```
aws_security_group.web_sg:
Creation complete after 6s

Apply complete!
Resources: 1 added, 0 changed, 0 destroyed.
```

Created Security Group ID:

`sg-08c4254f3fa86173e`

Name: `Final-Terraform-SG`

6. Conclusion

Successfully created a Security Group inside the custom VPC using Terraform. Configured inbound rules for SSH (22), HTTP (80), and HTTPS (443), and an outbound rule allowing all traffic.

Task 7: Create a network interface with an ip in the subnet that was created in step 4

1. Objective

Create a Network Interface (ENI) inside the public subnet and associate the Security Group with it.

2. Steps Used

1. Added the `aws_network_interface` resource to `main.tf`.
2. Referenced the existing public subnet.
3. Attached the existing Security Group.
4. Ran `terraform fmt`, `terraform validate`, and `terraform plan`.
5. Ran `terraform apply`.
6. Verified the ENI using AWS CLI.

3. Complete Code

Append this resource to your existing `main.tf`:

```
resource "aws_network_interface" "web_eni" {
  subnet_id       = aws_subnet.public_subnet.id
  security_groups = [aws_security_group.web_sg.id]

  tags = {
    Name = "Final-Terraform-ENI"
  }
}
resource "aws_vpc" "main" {
  cidr_block           = "10.0.0.0/16"
  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = {
    Name = "Final-Terraform-VPC"
  }
}
```

```
resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id

  tags = {
    Name = "Final-Terraform-IGW"
  }
}

resource "aws_route_table" "public_rt" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }

  tags = {
    Name = "Final-Terraform-Public-RT"
  }
}

resource "aws_subnet" "public_subnet" {
  vpc_id                  = aws_vpc.main.id
  cidr_block              = "10.0.1.0/24"
  availability_zone       = "us-east-1a"
  map_public_ip_on_launch = true

  tags = {
    Name = "Final-Terraform-Public-Subnet"
  }
}

resource "aws_route_table_association" "public_association" {
  subnet_id      = aws_subnet.public_subnet.id
  route_table_id = aws_route_table.public_rt.id
}

resource "aws_security_group" "web_sg" {
  name        = "Final-Terraform-SG"
```

```
  description = "Allow SSH, HTTP and HTTPS traffic"
  vpc_id      = aws_vpc.main.id

  ingress {
    description = "Allow SSH"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "Allow HTTP"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "Allow HTTPS"
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

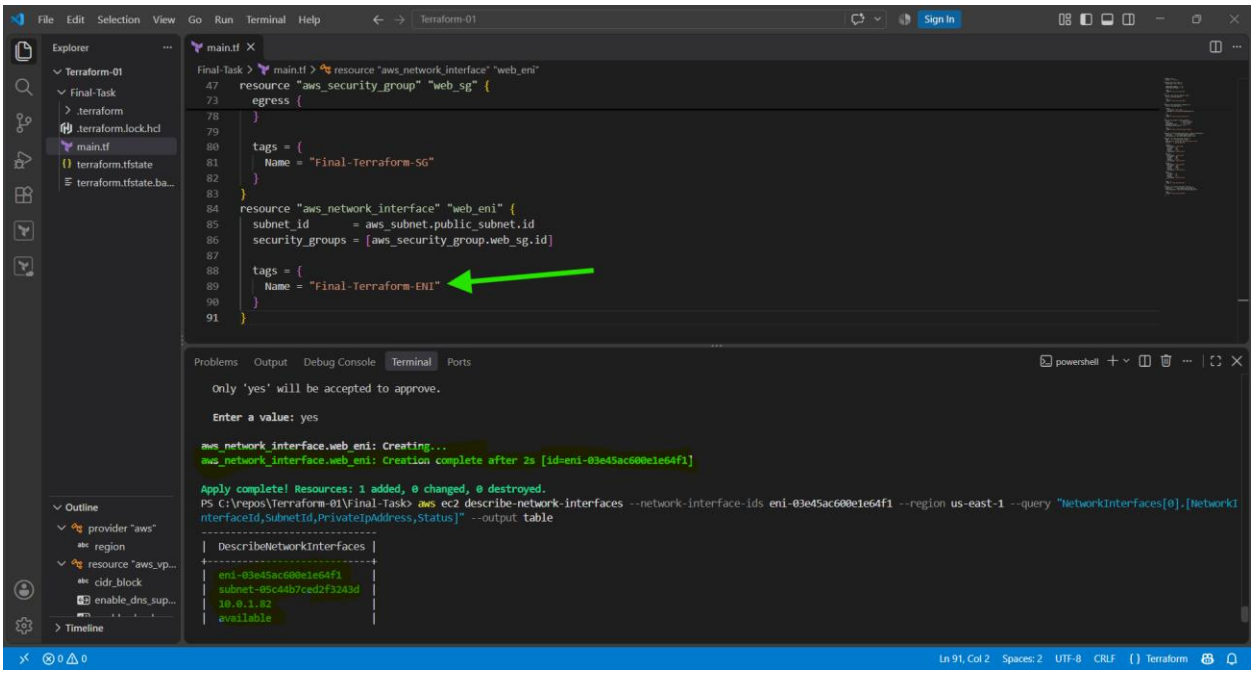
  tags = {
    Name = "Final-Terraform-SG"
  }
}
```

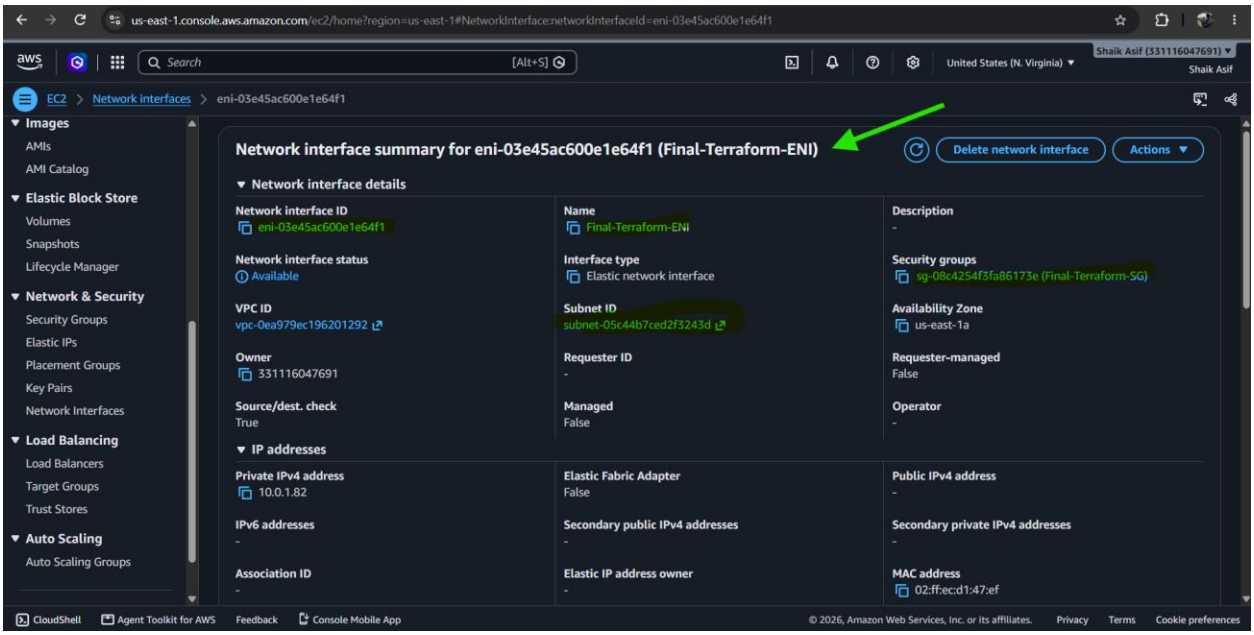
```
resource "aws_network_interface" "web_eni" {
  subnet_id       = aws_subnet.public_subnet.id
  security_groups = [aws_security_group.web_sg.id]

  tags = {
    Name = "Final-Terraform-ENI"
  }
}
```

4. Why Do We Need This?

- ENI acts as a virtual network card for an EC2 instance.
- It connects the EC2 instance to the VPC network.
- It holds a private IP address and Security Group.
- We create it separately because the next task associates an Elastic IP with this ENI.





5. Observation

- ENI created successfully.
- AWS assigned private IP `10.0.1.82`.
- ENI status: `available`.

6. Conclusion

Successfully created and verified a Network Interface in the public subnet. It is ready for Elastic IP association

Task 8:Assign an elastic IP to the network interface created in step 7

1. Objective

Allocate an Elastic IP and associate it with the Network Interface created in Task 7.

2. Steps Used

1. Added the `aws_eip` resource to `main.tf`.
2. Set the EIP domain to `vpc`.
3. Referenced the existing ENI.
4. Associated the EIP with private IP `10.0.1.82`.
5. Ran `terraform fmt`, `terraform validate`, `terraform plan`, and `terraform apply`.
6. Verified the Elastic IP using AWS CLI.

3. Complete Code

Add this resource after the ENI resource:

```
resource "aws_eip" "web_eip" {
  domain                    = "vpc"
  network_interface         = aws_network_interface.web_eni.id
  associate_with_private_ip = aws_network_interface.web_eni.private_ip

  tags = {
    Name = "Final-Terraform-EIP"
  }
}
```

Here is your complete cumulative `main.tf` code from Task 1 to Task 8:

```
resource "aws_vpc" "main" {
  cidr_block           = "10.0.0.0/16"
  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = {
    Name = "Final-Terraform-VPC"
  }
}

resource "aws_internet_gateway" "igw" {
```

```
  vpc_id = aws_vpc.main.id

  tags = {
    Name = "Final-Terraform-IGW"
  }
}

resource "aws_route_table" "public_rt" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }

  tags = {
    Name = "Final-Terraform-Public-RT"
  }
}

resource "aws_subnet" "public_subnet" {
  vpc_id                  = aws_vpc.main.id
  cidr_block              = "10.0.1.0/24"
  availability_zone       = "us-east-1a"
  map_public_ip_on_launch = true

  tags = {
    Name = "Final-Terraform-Public-Subnet"
  }
}

resource "aws_route_table_association" "public_association" {
  subnet_id      = aws_subnet.public_subnet.id
  route_table_id = aws_route_table.public_rt.id
}

resource "aws_security_group" "web_sg" {
  name        = "Final-Terraform-SG"
  description = "Allow SSH, HTTP and HTTPS traffic"
```

```
  vpc_id      = aws_vpc.main.id

  ingress {
    description = "Allow SSH"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "Allow HTTP"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "Allow HTTPS"
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "Final-Terraform-SG"
  }
}

resource "aws_network_interface" "web_eni" {
```

```
  subnet_id       = aws_subnet.public_subnet.id
  security_groups = [aws_security_group.web_sg.id]

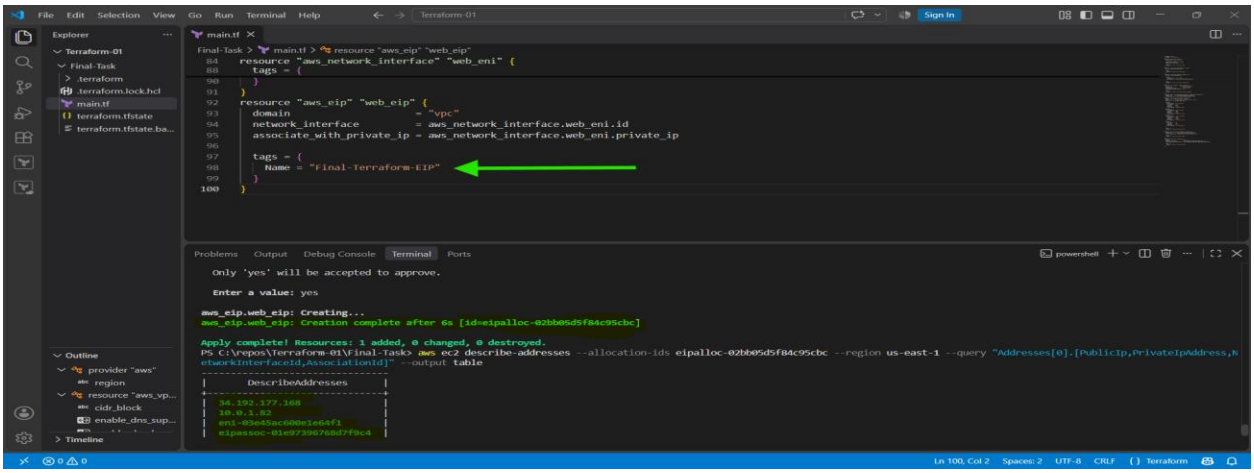
  tags = {
    Name = "Final-Terraform-ENI"
  }
}

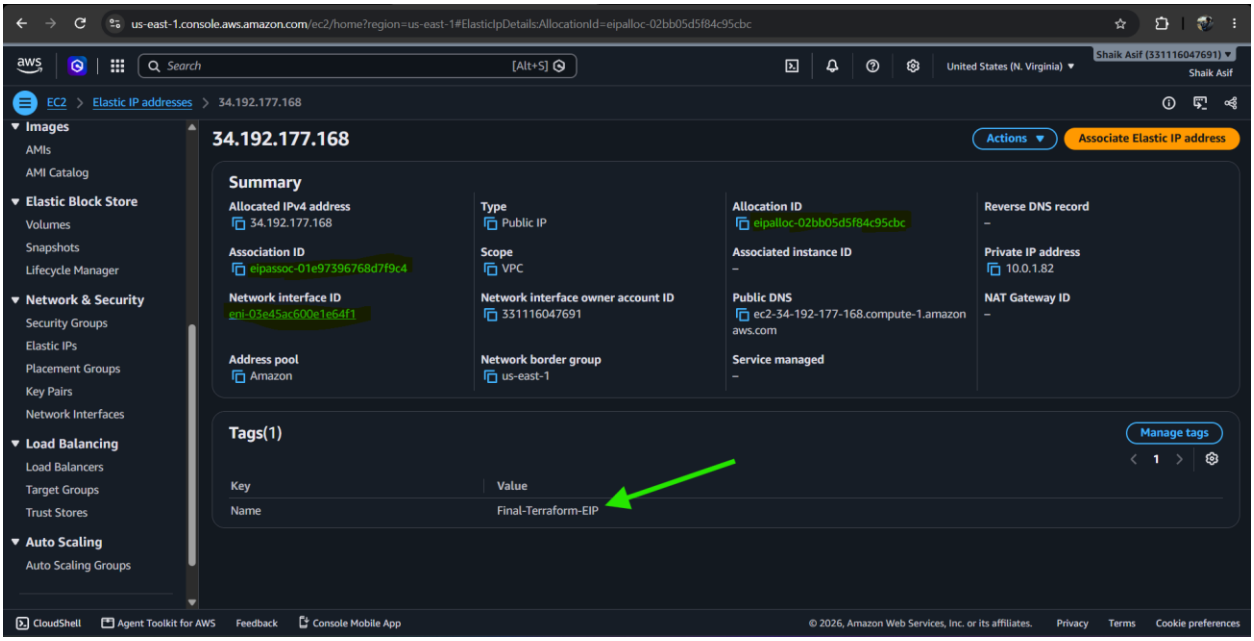
resource "aws_eip" "web_eip" {
  domain                    = "vpc"
  network_interface         = aws_network_interface.web_eni.id
  associate_with_private_ip = aws_network_interface.web_eni.private_ip

  tags = {
    Name = "Final-Terraform-EIP"
  }
}
```

4. Why Do We Need This?

- An Elastic IP provides a static public IPv4 address.
- It allows users to access the server from the internet.
- Unlike an automatically assigned public IP, an Elastic IP remains allocated to your AWS account until released.
- We associate it with the ENI so it can be used by the EC2 instance.





5. Observation

- Elastic IP allocated successfully.
- Public IP: `34.192.177.168`.
- Private IP: `10.0.1.82`.
- ENI association verified through AWS CLI.

6. Conclusion

Successfully allocated and associated an Elastic IP with the Network Interface. The network infrastructure is now ready for the EC2 web server.

Task 9: Create Ubuntu server and install/enable apache2

1. Objective

To provision a complete AWS infrastructure using Terraform in the `us-east-1` region.

The project includes:

1. Create a VPC.
2. Create an Internet Gateway.
3. Create a Custom Route Table.
4. Create a Public Subnet.
5. Associate the Subnet with the Route Table.
6. Create a Security Group allowing ports 22, 80 and 443.
7. Create a Network Interface.
8. Allocate and associate an Elastic IP.
9. Launch an Ubuntu EC2 instance and install Apache2 automatically.

2. Steps Used

Task	Resource	Result
1	VPC	Created
2	Internet Gateway	Created
3	Route Table	Created
4	Public Subnet	Created
5	Route Table Association	Associated
6	Security Group	Created

7	Network Interface	Created
8	Elastic IP	Associated
9	Ubuntu EC2 + Apache	Deployed

Commands used:

```
terraform init
terraform validate
terraform plan
terraform apply
```

During `terraform apply`, you entered `yes` to approve the infrastructure creation.

3. Complete Code

File: `main.tf`

Folder: `C:\repos\Terraform-01\Final-Task`

This is the complete cumulative code for all nine tasks.

```
provider "aws" {
  region = "us-east-1"
}

# Ubuntu AMI Data Source

data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]

  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-
server-*"]
```

```
  }

  filter {
    name   = "architecture"
    values = ["x86_64"]
  }

  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }
}

# Task 1: Create VPC
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"

  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = {
    Name = "Final-Terraform-VPC"
  }
}

# Task 2: Create Internet Gateway
resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id

  tags = {
    Name = "Final-Terraform-IGW"
  }
}

# Task 3: Create Custom Route Table
resource "aws_route_table" "public_rt" {
  vpc_id = aws_vpc.main.id

  route {
```

```
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }

  tags = {
    Name = "Final-Terraform-Public-RT"
  }
}

# Task 4: Create Public Subnet
resource "aws_subnet" "public_subnet" {
  vpc_id                  = aws_vpc.main.id
  cidr_block              = "10.0.1.0/24"
  availability_zone       = "us-east-1a"
  map_public_ip_on_launch = true

  tags = {
    Name = "Final-Terraform-Public-Subnet"
  }
}

# Task 5: Associate Subnet with Route Table
resource "aws_route_table_association" "public_association" {
  subnet_id      = aws_subnet.public_subnet.id
  route_table_id = aws_route_table.public_rt.id
}

# Task 6: Create Security Group
resource "aws_security_group" "web_sg" {
  name        = "Final-Terraform-SG"
  description = "Allow SSH, HTTP and HTTPS traffic"
  vpc_id      = aws_vpc.main.id

  ingress {
    description = "Allow SSH"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
```

```
  }

  ingress {
    description = "Allow HTTP"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "Allow HTTPS"
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "Final-Terraform-SG"
  }
}

# Task 7: Create Network Interface
resource "aws_network_interface" "web_eni" {
  subnet_id       = aws_subnet.public_subnet.id
  security_groups = [aws_security_group.web_sg.id]

  tags = {
    Name = "Final-Terraform-ENI"
  }
}
```

```
# Task 8: Allocate and Associate Elastic IP
resource "aws_eip" "web_eip" {
  domain                    = "vpc"
  network_interface         = aws_network_interface.web_eni.id
  associate_with_private_ip = aws_network_interface.web_eni.private_ip

  tags = {
    Name = "Final-Terraform-EIP"
  }
}

# Task 9: Create Ubuntu EC2 and Install Apache
resource "aws_instance" "web_server" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t3.micro"
  key_name      = "jenkins"

  primary_network_interface {
    network_interface_id = aws_network_interface.web_eni.id
  }

  user_data = <<-EOF
    #!/bin/bash
    apt-get update -y
    apt-get install -y apache2
    systemctl enable apache2
    systemctl start apache2
    echo "Welcome to Asif's Cloud Lab" > /var/www/html/index.html
  EOF

  tags = {
    Name = "Final-Terraform-Ubuntu-Server"
  }
}
```

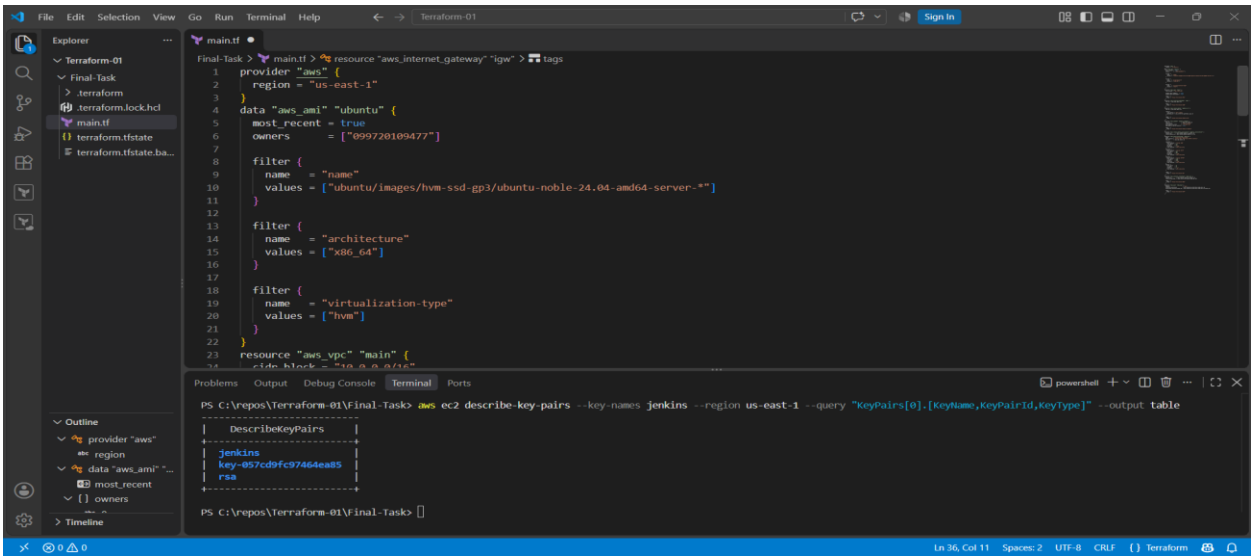
4. Why Do We Need This?

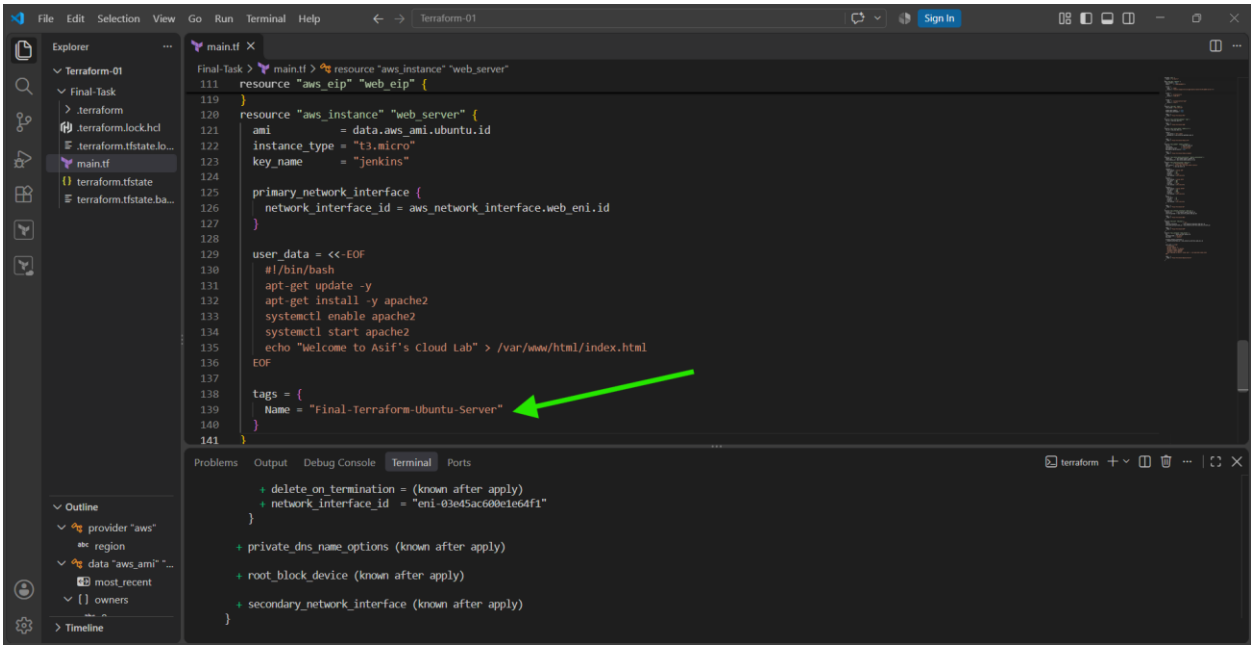
AWS Component	Purpose
VPC	Creates an isolated virtual network
Internet Gateway	Allows internet communication for the VPC
Route Table	Defines where network traffic should go
Public Subnet	Places resources in a subnet with internet routing
Route Association	Connects the subnet to its route table
Security Group	Controls inbound and outbound traffic
Network Interface	Provides network connectivity to EC2
Elastic IP	Provides a persistent public IPv4 address
EC2	Provides virtual computing capacity
AMI Data Source	Finds the latest matching Ubuntu image
User Data	Automates Apache installation during instance startup

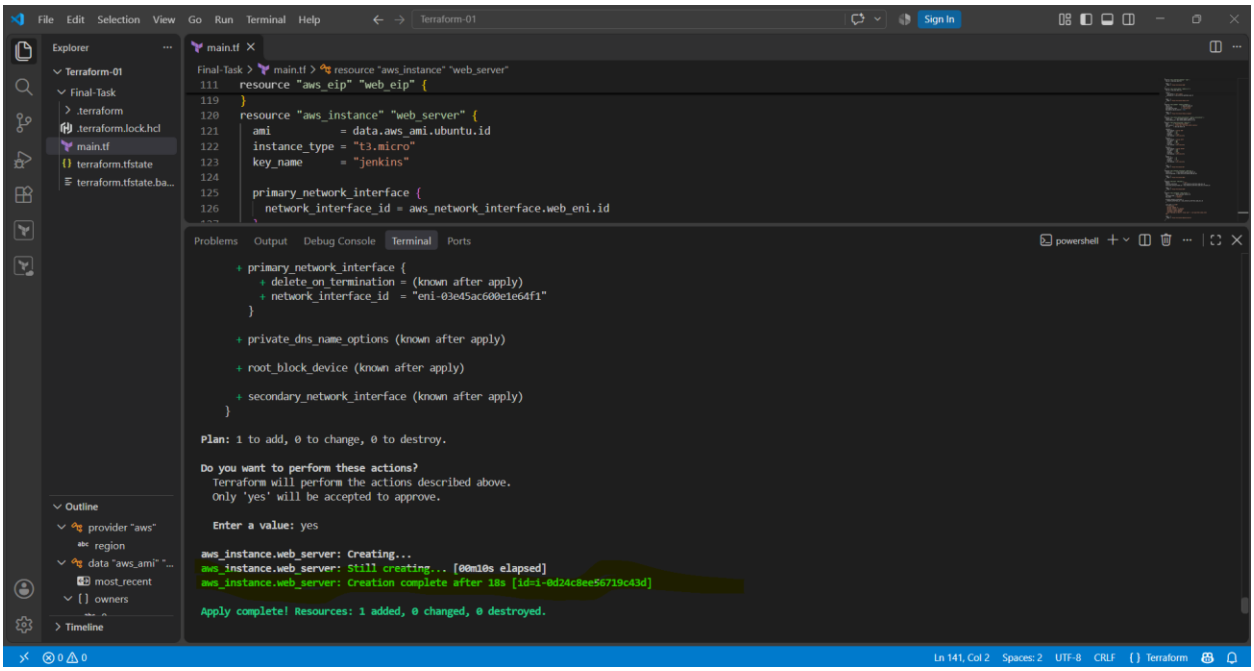
Important code explanations:

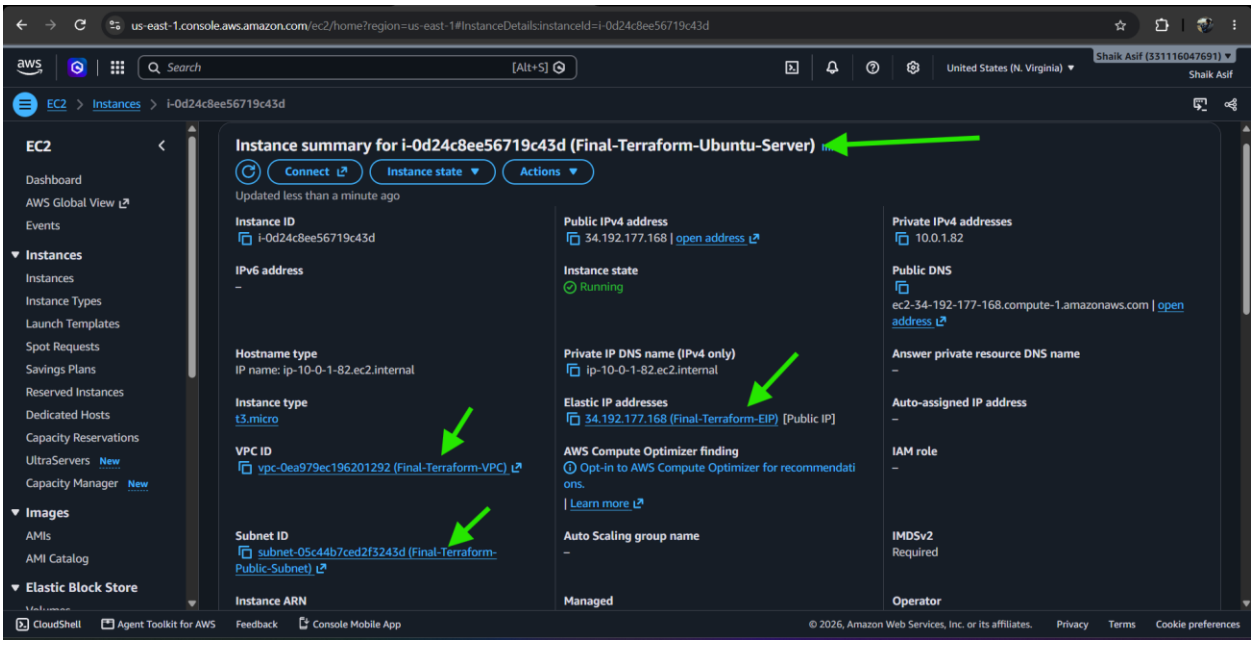
- `aws_vpc.main.id` – retrieves the VPC ID created by Terraform.
- `aws_internet_gateway.igw.id` – retrieves the Internet Gateway ID.
- `data.aws_ami.ubuntu.id` – retrieves the Ubuntu AMI ID.

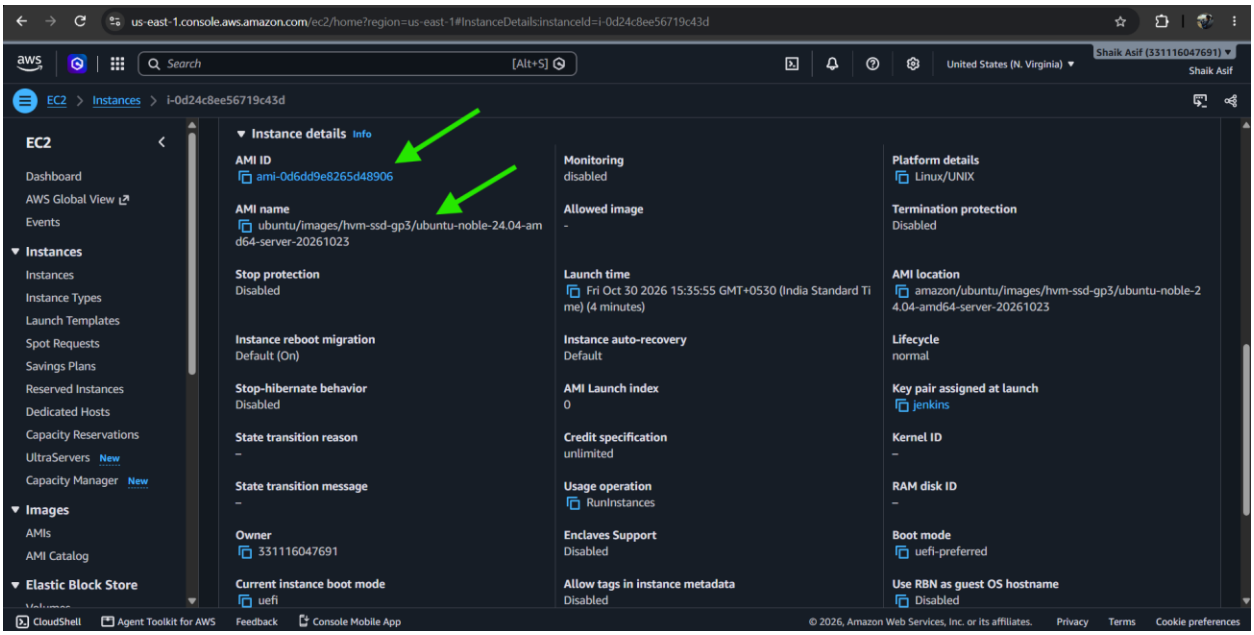
- `primary_network_interface` – attaches the previously created ENI as the EC2 primary network interface.
- `user_data` – executes the startup script automatically when the instance launches.






File Edit Selection View Go Run Terminal Help
Terraform-01
Sign In
Explorer
Terraform-01
Final-Task
.terraform
.terraform.lock.hcl
main.tf
terraform.tfstate
terraform.tfstate.ba...
main.tf
Final-Task > main.tf > resource "aws_instance" "web_server"
111 resource "aws_eip" "web_eip" {
119 }
120 resource "aws_instance" "web_server" {
121 ami = data.aws_ami.ubuntu.id
122 instance_type = "t3.micro"
123 key_name = "jenkins"
124
125 primary_network_interface {
126 network_interface_id = aws_network_interface.web_eni.id
Problems Output Debug Console Terminal Ports
powershell
+ primary_network_interface {
+ delete_on_termination = (known after apply)
+ network_interface_id = "eni-03e45ac600e1e64f1"
}
+ private_dns_name_options (known after apply)
+ root_block_device (known after apply)
+ secondary_network_interface (known after apply)
}
Plan: 1 to add, 0 to change, 0 to destroy.
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.
Enter a value: yes
aws_instance.web_server: Creating...
aws_instance.web_server: Still creating... [00m10s elapsed]
aws_instance.web_server: Creation complete after 18s [id=i-0d24c8ee56719c43d]
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
Outline
provider "aws"
region
data "aws_ami" "...
most_recent
owners
Timeline
Ln 141, Col 2 Spaces: 2 UTF-8 CRLF Terraform


us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#InstanceDetails:instanceId=i-0d24c8ee56719c43d
Search [Alt+S]
United States (N. Virginia)
Shaik Asif (331116047691)
Shaik Asif
EC2 > Instances > i-0d24c8ee56719c43d
EC2
Dashboard
AWS Global View
Events
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
UltraServers New
Capacity Manager New
Images
AMIs
AMI Catalog
Elastic Block Store
Instance summary for i-0d24c8ee56719c43d (Final-Terraform-Ubuntu-Server)
Connect
Instance state
Actions
Updated less than a minute ago
Instance ID
i-0d24c8ee56719c43d
IPv6 address
–
Hostname type
IP name: ip-10-0-1-82.ec2.internal
Instance type
t3.micro
VPC ID
vpc-0ea979ec196201292 (Final-Terraform-VPC)
Subnet ID
subnet-05c44b7ced2f3243d (Final-Terraform-Public-Subnet)
Instance ARN
Public IPv4 address
34.192.177.168 | open address
Instance state
Running
Private IP DNS name (IPv4 only)
ip-10-0-1-82.ec2.internal
Elastic IP addresses
34.192.177.168 (Final-Terraform-EIP) [Public IP]
AWS Compute Optimizer finding
Opt-in to AWS Compute Optimizer for recommendations.
Learn more
Auto Scaling group name
–
Managed
Private IPv4 addresses
10.0.1.82
Public DNS
ec2-34-192-177-168.compute-1.amazonaws.com | open address
Answer private resource DNS name
–
Auto-assigned IP address
–
IAM role
–
IMDSv2
Required
Operator
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences





Welcome to Asif's Cloud Lab

5. Observation

Final AWS Infrastructure

VPC CIDR

10.0.0.0/16

Subnet CIDR

10.0.1.0/24

EC2 instance

i-0d24c8ee56719c43d

Instance type

t3.micro

Private IP

10.0.1.82

Elastic IP

34.192.177.168

Apache webpage

Verified successfully

Welcome to Asif's Cloud Lab

HTTP access verified through the browser.

6. Conclusion

Successfully provisioned AWS infrastructure using Terraform.

Created a custom VPC, public subnet, Internet Gateway, route table, Security Group, Network Interface and Elastic IP. Launched an Ubuntu EC2 instance and automated Apache2 installation using Terraform `user_data`.

Finally, verified the Apache webpage using the Elastic IP.